SmartBIS 운영 수집 설계 (초안, GPS 없음)

> 상태: **초안**. 운수사 자료(`노선(사이텍).xlsx`, 11장), TAGO 노선 자료(`TAGO_노선도.xlsx`, 12장), 운수사–TAGO 노선 대응 결과(13장)를 반영했다. 확인 사항(10장)과 관측 결과가 정해지면 AGENTS.md의 P4로 옮긴다. 그 전에는 구현하지 않는다.

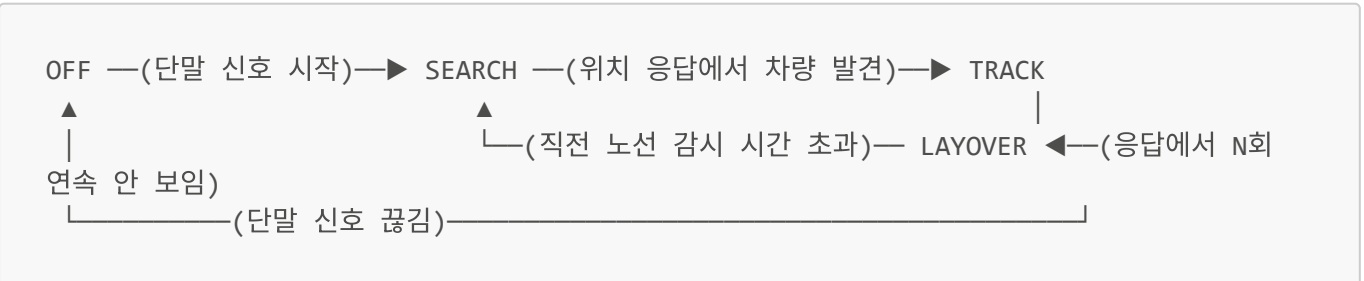
1. 전제

- 차량 단말에 GPS가 없다 → 차량 위치는 TAGO 위치 API(`getRouteAcctoBusLcList`)로만 알 수 있다
- 한도: API별 하루 10,000건(개발계정), 운영 예산 9,500건
- 운행 시간: 관내 06:15~21:00, 100번 05:30~. 관내 64개 노선(하루 편도 333회)과 100번(장성↔광주, 평일 84회·휴일 74회)
- 관내 노선의 48%(31개)는 하루 3회 이하, 89%(57개)는 10회 이하 운행한다
- 관내는 차량 18대가 **순번별 고정 운행표**(순번 1~10, ①~⑧)에 따라 여러 노선을 이어서 운행한다(관내총괄 시트의 숫자를 순번으로 해석함. 10장 1번 확인 필요)
- 정류장 간격은 평균 약 1.2분이다(관내 정류장 통과 하루 약 10,000회, 추정)
- TAGO 노선 ID는 **105개**로 확인했다(12장). 관내 88개(노선번호 48개)와 100번 계열 17개(이름 8개)다. 운수사 노선과 TAGO ID를 정류장 순서로 맞춘 결과, 관내 운행의 69%는 TAGO ID 1개로 확정되고 31%는 후보가 2~7개다(13장)
- 위치 API는 **노선 단위**로 조회한다: 1회 호출로 그 노선의 모든 차량(차량번호·위치)이 온다
- 도착정보 API 응답에는 차량번호가 없다 → 같은 노선·같은 방향 차량이 여러 대면 구분하기 어렵다
- TAGO 데이터 갱신 주기는 10~20초

2. 관리하는 것 / 관리하지 않는 것

대상	방식
차량–노선 배정	**관리하지 않는다.** 순번 운행표를 한 번 등록해 두고, 차량이 어느 순번인지는 운행 중 발견된 (노선, 시각)으로 자동 매칭한다
순번 운행표·노선 시간표	운수사 자료(관내총괄, 관내, 100번 평일/휴일)를 등록한다. 개편 때만 갱신한다
운행 준비 중인 차량	단말 신호(heartbeat)로 자동 판단한다 (단말 전원이 시동과 연동될 때)

3. 차량 상태



상태	뜻	호출
OFF	단말이 꺼져 있음	없음
SEARCH	단말은 켜졌는데 어느 노선인지 모름	스윕 대상
TRACK	노선을 알고 있음 (순번을 알면 이후 운행도 예측)	운행 중인 노선을 주기적으로 조회
LAYOVER	운행을 마치고 대기 중	순번을 알면 다음 출발 2분 전까지 호출 없음. 모르면 직전 노선만 감시(최대 30분)

4. 위치 API 호출 (①위치 수집 프로세스만 수행)

- **적용 범위:** ①위치 수집 프로세스만 TAGO 위치 API를 호출한다. 앱 API(⑤)와 화면(⑥)은 TAGO를 호출하지 않고 DB만 읽는다.
- **예산:** 일 9,500건(운영 예산). 개발계정 한도 10,000과 별도 설정값이다. 재시도는 최대 2회, 간격 10초이며 실패한 시도도 예산에 포함한다. 사용량은 `guard_usage_daily`에 기록하고 재시작 시 당일 값을 복원한다. 상한에 도달하면 그날 위치 API 호출을 멈추고 로그를 남긴다.
- **노선 그룹별 조회 방식:**
  - 100번 계열 17개: 05:30~21:00 상시 10분 주기 (약 1,581건)
  - 관내 88개: 배차 시각 주변의 **창(window)**에서만 조회한다. 창은 출발 15분 전부터 노선 도착 예정까지이며, 조회 주기는 5~10분이다. 창 계산은 관내 시간표 기준으로 별도 산출하며, 추정치는 약 2,000~3,000건이다.
  - 백업 스윕: 105노선 전체를 20~30분 주기로 1회씩 조회해 창 밖 운행과 예비차 투입을 잡는다.
- **예산 배분 원칙:** 100번 상시 조회 → 관내 창 → 백업 스윕 순으로 우선한다. 세 항목의 합계는 9,500건 이하, 재시도 여유 500건 이상을 유지한다. 예산이 부족하면 백업 스윕 주기를 먼저 늘린다.
- **호출 단위:** 노선 1회 호출로 응답의 모든 차량을 저장한다(`numOfRows=100`). 같은 노선에 우리 차량이 여러 대 있으면 한 번의 호출로 함께 갱신된다.
- **차량 상태(3장)와의 관계:** 창과 백업 스윕은 상태를 바꾸지 않는다. 상태 판정은 3장 규칙을 따르며, 창 안에서 차량이 확인되지 않으면 백업 스윕 결과로 SEARCH 전환을 판단한다.
- **패턴·배차 판정:** 스윕과 창의 관측 결과는 ③패턴·계획 판정이 전날 밤에 해석한다. 판정 결과인 `route_call_plan`만 ①이 읽는다. ①은 판정 로직을 알지 못한다.
- **관측 모드(현재):** 105노선을 10초 간격으로 순환한다. 일요일 분석 전까지 설정을 바꾸지 않는다. 이 절의 방식은 운영 전환 시점부터 적용한다.

5. 도착정보 API

- 정류장 간격이 짧아(평균 약 1.2분) 정류장마다 조회하면 한도를 넘는다(관내+100번 하루 약 13,000~14,000건, 추정)
- 조회하는 경우: ① 운행을 시작할 때 ② 다음 정류장까지 예상 소요가 3분 이상일 때 ③ 예정 시각이 지났는데 아직 통과하지 않았을 때
- 그 밖의 짧은 구간은 쌓인 위치 기록에서 학습한 **정류장 간 소요 시간**(시간대별 중앙값)으로 계산한다. 학습 전에는 마지막 도착 예정 시각에서 경과 시간을 빼서 보여준다
- 응답에서 `arrprevstationcnt`(남은 정류장 수)가 차량 위치와 맞는 값을 고른다. 관내 노선은 같은 노선·방향에 우리 차량이 동시에 1대인 경우가 대부분이라 혼동이 적다

6. 예산 추정 (운수사 자료 기준, 추정)

관내 차량 18대의 하루 실제 운행 시간 합계 R은 약 200시간이다(하루 운행거리 5,904km ÷ 평균 약 30km/h). 같은 노선에 우리 차량이 여러 대 있으면 1회로 함께 갱신되므로, 아래 위치 API 값은 상한이다.

항목	위치 API	도착정보 API
관내 추적 (120초 간격 / 90초 간격)	약 6,000 / 8,000건	–
100번 추적 (60초)	약 1,000~2,000건 (TAGO에서는 17개 ID로 나뉘지만 차량이 달리는 ID 1개만 조회하므로 운행 시간 기준으로는 같다)	–
출발 확인 (하루 운행 약 420회 × 1~2건. 관내만 보면 후보 전부 조회 시 약 570건, 학습 후 약 330건, 13장)	약 400~800건	–
도착정보 (5장 조건에 해당할 때만)	–	약 4,000~5,000건
합계 (관내 120초 기준)	**약 7,400~8,800건**	**약 4,000~5,000건**

- 예산 9,500건 안에 들지만 위치 API는 여유가 적다. 관내 추적 간격(120초)이 가장 큰 조절 수단이다
- 정류장마다 위치·도착정보를 확인하는 방식은 관내만으로도 API별 하루 약 10,000건이 들어 한도를 넘는다
- 100번 정류장 수와 도착정보 조회 비율은 추정이다. 관측으로 확인한다

7. 한도를 넘을 때의 선택지 (효과가 큰 순)

1. **운영계정 전환 + 트래픽 증가 신청:** 공공데이터포털에 활용사례를 등록하고 신청한다. 얼마나 늘려 주는지는 신청해 봐야 안다
2. **지자체 BIS 센터·운수사 위치 데이터를 직접 연계:** 버스의 BIS 단말이나 디지털 운행기록계에는 GPS가 있고, TAGO도 이 데이터를 모아서 제공한다. 직접 받으면 한도 문제가 사라진다
3. **외장 GPS 수신기(USB/블루투스):** 단말이 외부 장치 연결(USB OTG 등)을 지원하는지 확인이 필요하다
4. **정밀도 낮추기:** 2정류장마다 확인하고, 중간은 쌓인 기록에서 얻은 구간 소요 시간으로 추정한다. 호출은 약 절반이 되지만 정확도가 떨어진다
5. **도착정보로 통과 판정:** 같은 노선·방향에 동시에 1대만 다닐 때만 쓸 수 있다. 운수사 자료상 관내 노선 대부분이 이 조건에 해당한다(하루 10회 이하 노선 89%). 위치 API 사용은 줄지만 도착정보 사용은 늘어난다

8. 관측 모드 (먼저 할 일)

- **기간:** 평일 3일 + 토요일·일요일
- **설정:** 105개 노선 전체, 운행 시간 06:00~21:00, 위치 API 하루 9,500건(약 5.7초 간격) → 한 바퀴 약 10분
- **저장:** 우리 차량만이 아니라 응답에 나온 **모든 차량**(차량번호, 노선, 정류장 순번, 시각)
- **얻는 값:**
  - 순번 운행표와 실제 운행의 차이(지연·결행·순번 교체)
  - 대응표에서 후보가 여러 개인 16개 노선(관내 운행 103회)이 실제로 어느 TAGO ID로 잡히는지, 시간표에 대응되지 않는 TAGO ID 6개에 차량이 나타나는지(13장)
  - 정류장 간 실제 소요 시간(시간대별)
  - 실제 운행 시간의 합 R
  - 같은 노선·방향을 동시에 달리는 비율

- **확인 필요:** 지금 수집기가 응답의 모든 차량을 저장하는지(Codex에게 보고 요청)

9. 기존 코드와의 관계

- `TagoQuotaGuard`: 운영 값만 바꾸면 그대로 쓴다(`window`, `min-gap-ms`). 남은 예산에 따라 간격을 자동으로 조정한다
- `ArrivalService`의 2분 기준: 운영 방식에서는 위치가 2분 넘게 갱신되지 않을 수 있다 → "TRACK 상태이고 도착 예정 시각이 유효함" 기준으로 바꿔야 한다
- 다음 정류소 찾기: TAGO 순번은 노선 ID 안에서 0부터 끝까지 이어지고, 상·하행이 바뀌는 지점에서도 다시 시작하지 않는다(12장). 같은 방향 조건을 빼고 `route_id`와 `node_order + 1`로 찾아야 회차 지점의 404가 사라진다
- 방향 값: 31개 ID는 `updowncd=2`(방향 구분 없음)다. 방향이 UP/DOWN인 행만 쓰는 지금 조회 방식에서는 이 노선들(100번 계열 12개 ID 포함)이 항상 404가 될 수 있다
- 순번 0: 노선 정류소 순번이 0부터 시작한다. 수집기는 순번 1 미만 위치를 버리므로(308줄), 위치 API 순번도 0부터라면 기점에 있는 차량이 빠진다. 위치 API 응답으로 확인이 필요하다
- 테스트 단계(단말 3대, 10초 간격)와 운영 방식의 차이는 설정값과 상태 판단으로만 둔다. 테스트 전용 분기 코드는 만들지 않는다

10. 결정·확인 필요

1. 순번(1~10, ①~⑧)과 차량번호의 관계: 고정인지, 매일 바뀌는지. 바뀐다면 그날 배차를 기록한 자료(배차 일지 등)가 있는지
2. ①~⑧ 표시의 의미(영업소·차종 등)와 18대가 관내 전체 차량인지. 100번 운행 차량 수와 단말 설치 대상 여부
3. 관내 토·일·공휴일 운행표가 따로 있는지, 이 자료가 현재 기준인지(6번 노선에 '2022.02.04부터 운행' 메모가 있음)
4. 단말 전원이 시동과 연동되는지, 운영할 단말 대수
5. 운영계정 신청 가능 여부, 지자체 BIS 센터·운수사와 위치 데이터 연계 협의 가능 여부
6. 관측 모드를 진행할지, 진행한다면 언제 할지
7. 87번(사창–생촌, 하루 1회)은 TAGO에 없다 → TAGO로는 추적할 수 없다. 지금도 운행하는지, 단말 설치 대상인지
8. 100번: 운수사 자료는 장성↔광주 시간표 1개인데 TAGO에는 이름 8개(녹진·마령·비아·삼서·월곡·월정·장성·진원100), ID 17개가 있다. 시각별로 어느 경유 변형인지 알 수 있는 자료가 있는지
9. 운수사 자료 정리: 시간표의 `91-1`이 `관내 총괄 편집` 시트에서는 `91-2`로 되어 있다(같은 노선인지). 편집 시트의 12번 행은 경유 정류장이 비어 있다

11. 운수사 자료 분석 (`노선(사이텍).xlsx`)

시트	내용
관내	관내 64개 노선: 기점·종점, 거리, 편도 횟수, 기점/종점 출발 시각, 경유 정류장 순서
관내총괄	군민운수 관내 총괄 운행표: 방면별로 순번(1~10, ①~⑧)과 출발 시각
100번 평일 / 휴일	장성↔광주 100번, 23.6km, 평일 84회·휴일 74회, 05:30부터
터미널	장성공용버스터미널 주소·전화

- 순번 18개는 각각 하루 14~21회 출발하며 여러 방면을 이어서 운행한다. 첫 출발 06:15~07:20, 마지막 출발 17:50~21:00

- 예: 순번 ④는 10:00·12:00·14:00·16:00에 같은 방면으로 출발해 50분 뒤 돌아오는 2시간 주기를 반복한다
- 자료 정리 시 주의: 70·71·93번은 편도 횟수가 시각 개수의 절반으로 적혀 있다(왕복 기준으로 보임). 시트 합계 315회와 시각 기준 333회가 다르다. 출발 시각 옆 괄호(`(청룡)`, `(물메)` 등)는 경유 변형이다
- 관내총괄은 셀 병합과 방면 블록이 섞여 있어 자동 추출 결과 일부에 중복·누락이 있을 수 있다. 등록 전에 순번별 운행표를 운수사와 대조한다

12. TAGO 노선 자료 분석 (`TAGO_노선도.xlsx`, 2026-10-02 수집)

- 두 시트(노선별 경유정류소 5,339행 / 중복값 제거 105행)를 비교했다. 노선 ID 105개가 양쪽에 모두 있고, 정류장 순서와 정류장 수가 모두 일치한다. 정류장 순서가 같은 ID는 없다 → **105개는 중복이 아니라 서로 다른 노선 ID다**

구분	노선 이름	노선 ID
관내	48개 (1~95번)	88개
100번 계열	8개 (녹진·마령·비아·삼서·월곡·월정·장성·진원100)	17개
합계	56개	**105개**

구조	ID 수	특징
왕복형	74개	ID 하나에 기점→회차→기점이 모두 들어 있다. `updowncd` 0(상행)→1(하행)으로 한 번 바뀌고, 순번은 0부터 끝까지 이어진다
편도형	31개	`updowncd=2`(방향 구분 없음). 가는 방향과 오는 방향이 서로 다른 ID다(예: 20번 `...029` 터미널→사거리, `...030` 사거리→터미널). 종점에 도착하면 차량이 반대 방향 ID로 옮겨 간다

- 운수사 대응: 운수사 관내 64개 = 기본 번호 48개 + 세부 번호 16개(10-1 등). TAGO 이름 48개는 기본 번호와 모두 일치한다. 세부 번호와 TAGO 변형 ID는 개수가 꼭 맞지 않는다(예: 10번은 TAGO 7개 / 운수사 4개, 90번은 TAGO 4개 / 운수사 3개) → 이름이 아니라 **정류장 순서로 대응**해야 한다
- 운수사 87번은 TAGO에 없다
- 정류장 951개, ID당 정류장 10~113개(평균 51개), 좌표 누락 없음
- 수집 설계 반영: 스윕 대상은 105개 ID다. 편도형 노선에서 운행이 끝나면 반대 방향 ID를 먼저 확인한다(3장 LAYOVER의 "직전 노선"에 반대 방향 ID를 포함)

13. 운수사 노선 ↔ TAGO 노선 ID 대응 (`TAGO_노선도.xlsx`의 `관내 총괄 편집` 시트)

- 방법: 운수사 노선별 경유 정류장 순서(63개 노선)를 같은 번호의 TAGO ID 정류장 순서와 비교했다. 정류장 이름은 약칭이 섞여 있어(예: `북일면` ↔ `북일면사무소`) 비슷한 이름도 같은 정류장으로 보고, 순서가 일치하는 비율로 점수를 매겼다. 편도형은 가는·오는 ID를 이어 붙여 비교했다

결과	운수사 노선	관내 하루 운행
TAGO ID 1개로 확정	47개 (12번은 편집 시트에 정류장이 없지만 TAGO ID가 1개뿐이라 대응)	229회 (69%)

결과	운수사 노선	관내 하루 운행
후보 여러 개	16개: 10·10-2·10-3(후보 7개), 90(4개), 11·11-1·11-2·13·13-1·90-2(3개), 32·70·71·81·82·91(2개)	103회 (31%)
TAGO 없음	87번	1회

- 후보가 여러 개인 이유: 같은 번호의 TAGO ID끼리 정류장 순서가 94~99% 같다(예: 10번 7개 ID). 정류장 순서만으로는 구분할 수 없다 → 관측으로 "어느 시각의 운행이 어느 ID로 잡히는지" 확인한다
- 시간표에 대응되지 않는 관내 ID 6개: 2번 `...003`, 4번 `...006`, 61번 `...064`, 63번 `...067`, 70번 `...070`, 71번 `...073`. 지금은 운행하지 않는 옛 변형이거나 경유 변형(`(청룡)` 등)일 수 있다. 스윕 우선순위를 가장 낮게 두고, 관측에서 차량이 보이면 올린다
- 시간대별 운행 예정 ID(후보 전부 포함, 추정): 관내 88개 중 06시 36개, 07시 59개(최대), 낮 시간 42~54개, 19시 33개, 20시 13개, 21시 6개. 스윕은 지금 시간대의 운행 예정 ID부터 한다
- 이 대응은 자동 비교 결과다. 등록 전에 운수사와 대조한다